

SIMNet

Simulateur de Réseau Intelligent en Python

Rapport Technique de Groupe

Établissement	Institut Saint Jean — Yaoundé, Cameroun
Filière	INGÉNIEUR 3 SRT
Unité d'enseignement	Programmation Orientée Objet en Python
Examineur	M. Stephane Fedim
Année académique	2025 – 2026
Semestre	2

Composition du groupe

N°	Nom et Prénom	Rôle dans le projet
1	Tsala Yannick	Chef de projet — Module 1 (Modélisation réseau)
2	Selihe Emmanuel	Module 2 (Simulation de trafic)
3	Ngassa Kamga	Module 3 (Sécurité & Filtrage)
4	Aissatou Bintou Mohammadou	Module 4 (Surveillance & Rapports)
5	Sarr Salif	Module 5 (Interface console interactive)

Table des matières

1. Présentation du projet	3
2. Architecture générale — Vue d'ensemble	3
3. Module 1 — Modélisation du réseau	3
Validation IPv4	4
Saisir IP	4
Topologie et Liens	4
Module 2 — Simulation de trafic	4
4. Organisation des classes	4
Algorithme de routage : Dijkstra	5
5. Module 3 — Sécurité et filtrage	5
Règles de filtrage	5
Mécanisme de filtrage	6
Journal et authentification	6
6. Module 4 — Surveillance et rapports	6
Méthodes	6
Choix de conception	6
7. Module 5 — Interface console interactive	7
Fonctionnalités du menu	7
Structure du menu	8
8. Justification des choix de conception POO	8
Héritage	8
Encapsulation	8
Abstraction	8
Polymorphisme	8
9. Diagramme de classes (simplifié)	8
10. Difficultés rencontrées et solutions apportées	9
Conclusion	10

1. Présentation du projet

SIMNet est un simulateur de réseau d'entreprise entièrement orienté objet, développé en Python 3 dans le cadre de l'unité d'enseignement Programmation Orientée Objet (POO). Il modélise une infrastructure réseau complète, permettant de faire circuler des paquets de données, d'assurer la sécurité via un firewall, et de superviser le fonctionnement du réseau.

Le projet est structuré en 5 modules indépendants mais interconnectés, chacun géré par un membre du groupe. Cette organisation reflète une approche professionnelle du développement logiciel en équipe.

2. Architecture générale — Vue d'ensemble

Le projet est organisé selon la structure de fichiers imposée par le sujet :

```
project_isj_ing3_oop/
|
|-- src/
|   |-- equipments.py      # Classes des équipements réseau
|   |-- topologie.py       # Topologie, liens
|   |-- paquets.py        # Paquet, simulation de trafic
|   |-- securite.py        # Firewall, règles, journal
|   |-- moniteur.py        # Moniteur réseau, rapport
|   '-- main.py           # Point d'entrée, menu interactif
|
|-- rapport.pdf            # Rapport technique du groupe
'-- README.md              # Description et instructions de
    lancement
```

Chaque fichier correspond à un module fonctionnel. La classe **Topologie** est le pivot central utilisé par tous les modules : elle stocke tous les équipements et tous les liens, et est passée en référence aux autres composants.

3. Module 1 — Modélisation du réseau

Le Module 1 constitue la **fondation de tout le projet**. Il définit les classes représentant les équipements réseau et la topologie qui les relie.

Hiérarchie des équipements

Classe	Attributs spécifiques	Méthodes spécifiques
--------	-----------------------	----------------------

Equipement (mère)	nom, adresse_ip, marque, statut	activer(), desactiver(), afficher()
Routeur	table_routage (dict)	ajouter_route(), supprimer_route()
Switch	vlan (list)	ajouter_vlan(), supprimer_vlan()
Serveur	services (list)	ajouter_service(), retirer_service()
Firewall	regles (list), journal (list)	ajouter_regle(), filtrer(), afficher_journal()
PointAccesWifi	ssid (str), canal (int)	afficher()
Terminal	(aucun)	afficher()

Validation IPv4

Une fonction utilitaire **valider_ipv4()** vérifie que chaque adresse IP entrée respecte le format IPv4 (4 octets séparés par des points, chaque octet entre 0 et 255). Elle est appelée dans le constructeur d'Equipement.

Saisir IP

Une fonction utilitaire Saisir_ip() tiens en compte la validation de chaque adresse IP tout en vérifiant qu'il n'y a pas de redondance d'adresse dans le trafic ,topologie

Topologie et Liens

La classe **Lien** représente la connexion physique entre deux équipements, caractérisée par une bande passante (Mbps) et une latence (ms). La classe **Topologie** est le conteneur central : elle maintient la liste de tous les équipements et de tous les liens, et expose des méthodes pour les ajouter, les supprimer, les rechercher et les afficher.

Module 2 — Simulation de trafic

4. Organisation des classes

Pour respecter les principes de la Programmation Orientée Objet, j'ai décidé de découper le module en 4 fichiers distincts, chacun contenant une classe bien précise. Chaque classe a une responsabilité unique et ne fait qu'une seule chose.

Classe	Rôle	Méthodes principales
Protocole	Constantes TCP/UDP/ICMP + validation	est_valide()

Paquet	Représente un paquet réseau	<code>__init__()</code> , <code>afficher()</code> , <code>__str__()</code>
Statistiques	Compteurs et historique du trafic	<code>enregistrer()</code> , <code>afficher()</code> , <code>vers_texte()</code>
SimulateurTrafic	Moteur Dijkstra + transmission saut/saut	<code>calculer_chemin()</code> , <code>transmettre()</code>

Algorithme de routage : Dijkstra

Pour calculer le chemin entre deux équipements, j'ai choisi l'algorithme de Dijkstra. Ce choix se justifie de la façon suivante :

- BFS (parcours en largeur) : ignoré car il ne tient pas compte des poids des liens. Il trouverait le chemin avec le moins de sauts, pas le chemin le plus rapide.
- Bellman-Ford : ignoré car il est conçu pour les graphes avec des poids négatifs, ce qui n'est pas notre cas (la latence est toujours positive).
- Dijkstra : retenu car il garantit le chemin de latence minimale dans un graphe à poids positifs. Il est également simple à implémenter sans bibliothèque externe.

Dans notre implémentation, le coût d'un lien est sa latence en millisecondes. L'algorithme minimise donc le temps de transit total du paquet. On calcule aussi la bande passante minimale sur le chemin (goulot d'étranglement) pour estimer le débit réel.

5.Module 3 — Sécurité et filtrage

Le Module 3 gère la sécurité du réseau via la classe **Firewall** étendue dans le fichier **securite.py**.

Règles de filtrage

Chaque règle est un dictionnaire Python contenant une action (bloquer / autoriser) et des critères de filtrage optionnels :

Critère	Type	Description
action	str	"bloquer" ou "autoriser"
ip_source	str / None	Filtrage par adresse IP source
protocole	str / None	Filtrage par protocole (TCP, UDP, ICMP)
port	int / None	Filtrage par port destination

Mécanisme de filtrage

La méthode **filtrer(paquet)** parcourt les règles dans l'ordre. Dès qu'une règle correspond au paquet (tous les critères non-None correspondent), la décision est appliquée et enregistrée dans le journal avec horodatage. Si aucune règle ne correspond, le paquet est autorisé par défaut.

Journal et authentification

Toute décision du firewall est journalisée avec un horodatage généré via le module **datetime** de la bibliothèque standard. L'accès à la configuration du firewall (ajout / suppression de règles) est protégé par un mécanisme d'authentification login / mot de passe géré en mémoire.

6.Module 4 — Surveillance et rapports

Classe principale du module. Elle collecte et centralise toutes les informations de surveillance du réseau.

Méthodes

Méthode	Rôle
paquet_transmis(nom_equipement)	Incrémente le compteur de paquets transmis pour un équipement
paquet_perdu(nom_equipement)	Incrémente le compteur de paquets perdus pour un équipement
ajouter_paquet(paquet)	Ajoute un paquet à l'historique (max 10)
enregistrer_lien(nom_lien, octets)	Enregistre le trafic passé sur un lien réseau
mettre_a_jour_equipements(liste)	Met à jour les listes actifs/inactifs
afficher_statistiques()	Affiche les statistiques dans la console
generer_rapport()	Exporte le rapport complet dans rapport_simnet.txt

Choix de conception

Utilisation de collections.deque

L'historique des paquets utilise une deque avec maxlen=10 de la bibliothèque standard Python. Ce choix est justifié car la deque gère automatiquement la limite de 10 éléments sans suppression manuelle, contrairement à une liste classique.

Suivi par équipement

Les statistiques sont suivies par équipement via un dictionnaire stats_equipements. Cela permet d'identifier précisément quel équipement transmet ou perd le plus de paquets, essentiel pour le diagnostic réseau.

Génération de rapport avec with open

La méthode generer_rapport() utilise le gestionnaire de contexte with open(...) qui garantit la fermeture automatique du fichier même en cas d'erreur, conformément aux bonnes pratiques Python.

Horodatage avec datetime

Chaque rapport est horodaté grâce au module datetime de la bibliothèque standard, permettant de tracer la date et l'heure exacte de génération.

7.Module 5 — Interface console interactive

Le Module 5 est le point d'entrée du simulateur, implémenté dans **main.py**. Il propose un menu interactif en boucle permettant à l'utilisateur de contrôler entièrement le simulateur sans modifier le code source.

Fonctionnalités du menu

Option	Action
1	Ajouter un équipement au réseau
2	Supprimer un équipement du réseau
3	Activer / Désactiver un équipement
4	Gérer un équipement (routes / vlans / services)
5	Ajouter un lien entre deux équipements
6	Supprimer un lien
7	Afficher la topologie complète
8	Envoyer un paquet et visualiser son parcours
9	Afficher les statistiques
10	Afficher l'historique des paquets
11	Configurer le Firewall
12	Consulter le journal du firewall
13	Générer un rapport
0	Quitter proprement le simulateur

Structure du menu

Le menu est implémenté avec une boucle **while True** et une structure **if / elif / else** et **match case**. Chaque option appelle les méthodes des classes correspondantes. La saisie utilisateur est récupérée avec **input()** et validée avant traitement.

8. Justification des choix de conception POO

Héritage

Nous avons utilisé l'héritage pour modéliser la hiérarchie naturelle des équipements réseau. La classe mère **Equipement** regroupe les attributs communs (nom, adresse_ip, marque, statut) et les méthodes de base (activer, désactiver, afficher). Les sous-classes héritent de tout cela et y ajoutent leurs spécificités. Ce choix évite la duplication de code et facilite la maintenance : une modification dans Equipement se propage automatiquement à toutes les sous-classes.

Encapsulation

Chaque classe encapsule ses propres données et les méthodes qui les manipulent. Par exemple, la table de routage d'un Routeur n'est modifiable que via **ajouter_route()** et **supprimer_route()**. La validation de l'adresse IPv4 est encapsulée dans le constructeur d'Equipement. L'utilisateur du simulateur n'a pas à connaître les détails internes des classes.

Abstraction

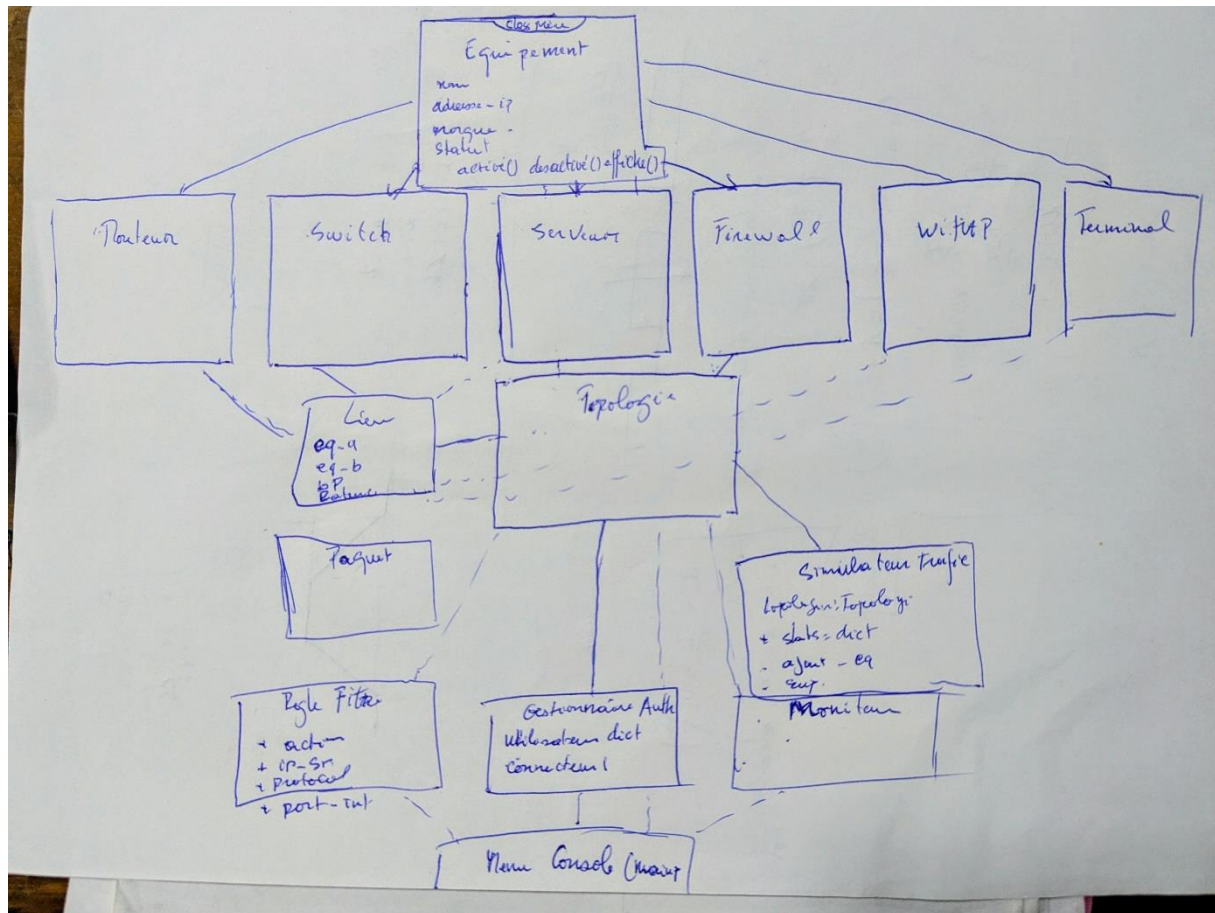
La classe Equipement représente une abstraction d'un équipement réseau réel. Un routeur physique est un appareil très complexe, mais pour notre simulation, nous n'avons retenu que les caractéristiques pertinentes : nom, IP, marque, statut et table de routage. Cette abstraction simplifie le modèle tout en restant fidèle au comportement essentiel de chaque équipement.

Polymorphisme

La méthode **afficher()** est définie dans Equipement et redéfinie dans chaque sous-classe (surcharge). Chaque équipement affiche ses informations de base via **super().afficher()** puis ajoute ses informations spécifiques. Cela permet de traiter une liste mixte d'équipements de façon uniforme.

9. Diagramme de classes (simplifié)

Le diagramme ci-dessous représente les relations entre les principales classes de SIMNet. L'indique l'héritage. La ligne pointillée indique l'association.



10. Difficultés rencontrées et solutions apportées

Reconstruction du chemin dans Dijkstra

Solution : j'ai utilisé un dictionnaire predecesseurs qui mémorise, pour chaque équipement, par quel équipement on est passé pour l'atteindre. À la fin, on remonte ce dictionnaire depuis la destination jusqu'à la source, puis on inverse la liste pour avoir le chemin dans le bon sens.

Gestion des équipements inactifs

Solution : dans la méthode `_construire_graphe()`, on vérifie le statut actif des deux équipements de chaque lien avant de l'ajouter au graphe. Si l'un d'eux est inactif, le lien est simplement ignoré.

Calcul du débit simulé

Solution : on convertit la taille du paquet en bits, on divise par le temps de transit converti en secondes pour obtenir un débit théorique en Mbps. On prend ensuite le minimum entre ce débit théorique et la bande passante du lien le plus faible sur le chemin (goulot d'étranglement).

Dépendance circulaire entre les modules

Solution : le SimulateurTrafic ne connaît pas la classe Topologie. Il reçoit simplement un objet topologie en paramètre dans son constructeur. Tant que cet objet expose les attributs liens et équipements, le simulateur fonctionne. C'est le principe du couplage faible.

Fermeture sécurisée du fichier

Gestionnaire de contexte with open(...)

Horodater le rapport

Module datetime de la bibliothèque standard

Journal du firewall avec horodatage : enregistrer la date et l'heure de chaque décision.

Utilisation du module datetime de la bibliothèque standard Python. Datetime

Interface console : gérer les saisies incorrectes de l'utilisateur sans planter le programme.

Encapsulation de chaque saisie dans un bloc try/except. Validation des entr

Conclusion

Le projet SIMNet nous a permis de mettre en pratique l'ensemble des concepts de la Programmation Orientée Objet vus en cours : héritage, encapsulation, abstraction et polymorphisme. La conception modulaire adoptée a facilité le travail en équipe et la maintenance du code.

L'approche retenue — partir d'un diagramme de classes papier avant d'écrire la moindre ligne de code — s'est révélée essentielle pour éviter les refactorisations coûteuses en fin de semaine.

SIMNet est un simulateur simple mais bien conçu, fonctionnel sur l'ensemble des 5 modules demandés, avec un code organisé, documenté et testé.